Lab 4 — L'extraction

Un module quitte le monolithe : microservice FastAPI, compose et carte

Dr. El Hadji Bassirou TOURE
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop

Année universitaire 2026–2027

Résumé

La direction de SénResto veut faire évoluer le calcul d'itinéraires avec l'écosystème géospatial de Python, et déployer ces évolutions sans redéployer le reste. Ce lab répond par l'**extraction** : le module `livraison` quitte le monolithe et devient **livraison-service**, un microservice **FastAPI**, selon le pattern **Strangler Fig** — le nouveau pousse autour de l'ancien, l'application fonctionne à chaque instant. Côté monolithe, `AssignationLivreurs` garde exactement la même API mais devient une *façade* qui traverse le réseau — `commandes` ne remarquera rien. **docker compose** orchestre les quatre conteneurs, une **carte Leaflet de Dakar** suit le livreur en temps réel, et la séance se clôt sur une expérience fondatrice : nous **tuerons le service** pour regarder mourir la confirmation de commande — le cliffhanger du Lab 5.

Benefice

À l'issue de ce lab, vous saurez :
- justifier (ou refuser) une extraction : technologie, déploiement, dimensionnement — et défendre des non-décisions ;
- extraire un module en **Strangler Fig**, sans interruption de service, grâce aux frontières du modulith ;
- écrire un microservice **FastAPI** et le faire dialoguer avec Spring via **RestClient** (timeouts, traduction d'erreurs) ;
- orchestrer un système multi-conteneurs avec **docker compose**, et brancher une carte **Leaflet** sans clé API.

Point d'attention

Prérequis avant la séance :
- votre dépôt au tag `lab4-depart` (= `lab3-final`) ; `./mvnw clean test` vert : **10 juges** ;
- **Python 3.12+** installé (`python3 -version`) ; Docker Desktop fonctionnel ;
- connexion internet (pip : fastapi, uvicorn ; npm : react-leaflet).

Table des matières

1 Le constat : une bonne raison, un bon candidat **3**
1.1 Exercice (10 minutes, en groupe) 3

2 Etape 1 — Le service Python nait **4**
2.1 Le contrat d'abord 4
2.2 main.py : le service complet 4
2.3 Premier vol, au curl 5

3 Etape 2 — La facade : le monolithe traverse le reseau **5**
3.1 Le Strangler en action : deux implementations cohabitent 5
3.2 Les profils : l'interrupteur du Strangler 6

4 Etape finale — Compose, la carte... et la panne **7**
4.1 docker compose : quatre conteneurs, un geste 7
4.2 La carte de Dakar 7
4.3 L'experience finale : la panne 8

5 Recapitulatif : qui a fait quoi **9**

6 L'ADR du jour **9**

7 Livrable et criteres de reussite **9**

8 Vers le Lab 5 : ne plus mourir ensemble **9**

1 Le constat : une bonne raison, un bon candidat

Intuition

On ne déménage pas parce que déménager est à la mode : on déménage parce que le nouveau logement offre ce que l'ancien ne peut pas offrir. Un microservice, c'est pareil — et la première tâche de l'architecte est de vérifier que la raison est réelle et que le candidat est le bon.

1.1 Exercice (10 minutes, en groupe)

Pour chacun des sept modules, répondez : une autre technologie lui serait-elle utile ? Un déploiement indépendant ? Un dimensionnement indépendant ? Ses données sont-elles partagées ? Concluez : quel est LE candidat — et pourquoi les six autres ne le sont pas. Gardez vos notes : c'est la matière première de l'ADR n°4, le premier qui devra défendre des *non-décisions*.

Point cle a retenir

`livraison` coche toutes les cases : frontière nette (Lab 3), API étroite (`assigner`, `liberer`, `tous`), données propres (les livreurs), raison technologique (l'écosystème géo de Python) et raison de déploiement (évolutions fréquentes demandées). C'est l'extraction *la moins chère* avec le bénéfice *le plus clair*.

Concept fondamental

Strangler Fig (Fowler). Plutôt que le « big bang » (tout réécrire, tout basculer, prier), le nouveau système **pousse autour de l'ancien** : on construit le service à côté, on fait basculer la façade, l'ancien code meurt quand plus rien ne l'appelle. À chaque commit, l'application fonctionne. Notre monolithe modulaire rend l'opération presque chirurgicale : la frontière et l'API existent déjà.

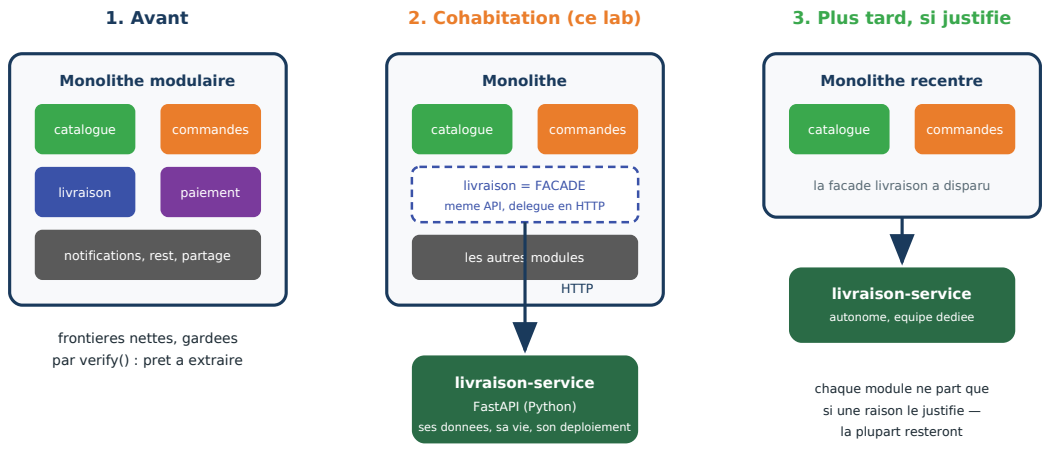


FIGURE 1 – Le figuier étrangleur : trois phases, zéro interruption.

2 Etape 1 — Le service Python nait

Qui pilote cette section ?

L'Appariteur pilote le nouveau service — c'est son jour de gloire. **Le Notaire** relit le contrat HTTP contre l'API du module (mêmes règles, mêmes messages). **Les autres** préparent l'étape 2.

À la racine du dépôt (à côté de `backend/` et `frontend/`), créez `livraison-service/` :

```
livraison-service/
|-- main.py             # l'application FastAPI complete
|-- requirements.txt    # fastapi + uvicorn
\-- Dockerfile          # pour l'etape finale
```

2.1 Le contrat d'abord

Le service traduit l'API du module en HTTP — vocabulaire et messages **strictement identiques** :

```
POST   /assignations    {"quartier": "Medina",
                         "restaurant_quartier": "Ouakam"}
       -> 200 {"livreur_id": 1, "livreur_nom": "...", "eta_minutes": 43}
       -> 409 {"erreur": "Aucun livreur disponible pour le moment, ..."}
DELETE /assignations/{livreur_id}     # liberer
GET    /livreurs                      # liste complete
GET    /livreurs/{id}/position        # position simulee (pour la carte)
```

2.2 main.py : le service complet

Le cœur du service — remarquez que `QuartiersDakar` et le calcul d'ETA **déménagent en Python** : le service possède désormais ce savoir, et ses données livreurs vivent en mémoire chez lui (*database per service*, version simple) :

```python
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import math, time

app = FastAPI(title="SenResto - Service Livraison")
app.add_middleware(CORSMiddleware, allow_origins=["*"],
                   allow_methods=["*"], allow_headers=["*"])

QUARTIERS = {  # l'ex-QuartiersDakar, desormais propriete du service
    "Plateau":     {"frais": 1000, "lat": 14.667, "lng": -17.436},
    "Medina":      {"frais": 1000, "lat": 14.677, "lng": -17.451},
    # ... les 7 quartiers, memes valeurs qu'au Lab 3 ...
}

LIVREURS = {   # ses donnees, chez lui (memes valeurs que data.sql)
    1: {"nom": "Ibrahima Ndiaye", "telephone": "771112233",
        "lat": 14.690, "lng": -17.446, "disponible": True},
    2: {"nom": "Cheikh Sarr", "telephone": "772223344",
        "lat": 14.710, "lng": -17.470, "disponible": True},
}
COURSES = {}   # livreur_id -> details de la course en cours (pour la carte)

class DemandeAssignation(BaseModel):
    quartier: str                          # celui du client
    restaurant_quartier: str | None = None # celui du restaurant

def distance_km(lat1, lng1, lat2, lng2):
    d_lat, d_lng = math.radians(lat2 - lat1), math.radians(lng2 - lng1)
    a = (math.sin(d_lat / 2) ** 2 + math.cos(math.radians(lat1))
         * math.cos(math.radians(lat2)) * math.sin(d_lng / 2) ** 2)
    return 6371.0 * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))

@app.post("/assignations")
def assigner(demande: DemandeAssignation):
    cible = QUARTIERS.get(demande.quartier)
    if cible is None:
        raise HTTPException(400, detail="Quartier non livrable")
    candidats = [i for i, l in LIVREURS.items() if l["disponible"]]
    if not candidats:
        raise HTTPException(409, detail="Aucun livreur disponible "
                                "pour le moment, reessayez plus tard")
    livreur_id = min(candidats)          # meme regle : le premier par id
    livreur = LIVREURS[livreur_id]
    # Le trajet REEL : livreur -> restaurant (recuperer le plat) -> client.
    resto = QUARTIERS.get(demande.restaurant_quartier or "")
    points = [(livreur["lat"], livreur["lng"])]
    if resto is not None:
        points.append((resto["lat"], resto["lng"]))
    points.append((cible["lat"], cible["lng"]))
```

```
    d = sum(distance_km(*points[i], *points[i + 1])
            for i in range(len(points) - 1))
    eta = math.ceil(d / 18.0 * 60.0) + 10
    livreur["disponible"] = False
    COURSES[livreur_id] = {"points": points,
                           "debut": time.time(), "duree_s": eta * 60}
    return {"livreur_id": livreur_id, "livreur_nom": livreur["nom"],
            "eta_minutes": eta}
```

Complétez sur le même modèle : `DELETE /assignations/{id}` (le livreur redevient disponible, sa course s'efface), `GET /livreurs`, et la position simulée — une interpolation *par morceaux* le long des jambes du trajet (livreur → restaurant → client), au prorata de la distance — le squelette complet est fourni dans le zip :

```
@app.get("/livreurs/{livreur_id}/position")
def position(livreur_id: int):
    course = COURSES.get(livreur_id)
    # ... 404 si livreur inconnu ; point d'attache si pas en course ...
    avancement = min(1.0, (time.time() - course["debut"]) / course["duree_s"])
    # interpolation PAR MORCEAUX sur course["points"], au prorata
    # de la distance parcourue (voir le corrige) :
    #   d_parcourue = avancement * d_totale, puis on avance de jambe
    #   en jambe jusqu'a trouver celle qui contient d_parcourue.
    return {"latitude": lat, "longitude": lng, "avancement": round(avancement, 3)}
```

2.3 Premier vol, au curl

```
cd livraison-service
python3 -m venv .venv && source .venv/bin/activate
pip install fastapi uvicorn
uvicorn main:app --port 8001 --reload

# Dans un second terminal :
curl -s -X POST localhost:8001/assignations \
     -H "Content-Type: application/json" -d '{"quartier": "Medina"}'
# {"livreur_id":1,"livreur_nom":"Ibrahima Ndiaye","eta_minutes":16}
curl -s localhost:8001/livreurs/1/position     # elle bouge !
curl -s -X POST ... une 3e assignation -> 409  # plus de livreur
```

Jalon Git

Le service répond au curl sur les quatre endpoints, le 409 au bon moment :

`git tag lab4-etape1 && git push origin lab4-etape1`

3 Etape 2 — La facade : le monolithe traverse le reseau

Qui pilote cette section ?

Le Notaire pilote l'adaptateur HTTP. **L'Huissier** pilote les profils Spring. **Le Doyen** vérifie l'invariant sacré : pas une ligne modifiée dans `commandes`.

3.1 Le Strangler en action : deux implementations cohabitent

C'est ici que le pattern prend chair. `AssignationLivreurs` devient une **interface** (l'API publique ne change pas d'une signature), et *deux* implémentations vivent côte à côte dans `interne/` :

```
// livraison/AssignationLivreurs.java  -- devient une interface
public interface AssignationLivreurs {
    Optional<Assignation> assigner(String quartier);
    void liberer(long livreurId);
    List<Map<String, Object>> tous();
}
```

```
// livraison/interne/AssignationLocale.java -- l'ANCIENNE implementation
// (l'ex-classe du Lab 3, renommee ; JDBC + CalculateurEta inchanges)
@Profile("!distribue")
@Component
class AssignationLocale implements AssignationLivreurs { /* ... */ }
```

```java
// livraison/interne/AdaptateurLivraisonHttp.java -- la NOUVELLE
@Profile("distribue")
@Component
class AdaptateurLivraisonHttp implements AssignationLivreurs {

    private final RestClient client;

    AdaptateurLivraisonHttp(@Value("${livraison.url}") String url) {
        this.client = RestClient.builder()
                .baseUrl(url)
                .requestFactory(usineAvecTimeouts())   // 2 s connexion, 3 s lecture
                .build();
    }

    @Override
    public Optional<Assignation> assigner(String quartier) {
        try {
            Map<String, Object> corps = client.post().uri("/assignations")
                    .body(Map.of("quartier", quartier))
                    .retrieve().body(Map.class);
            return Optional.of(new Assignation(
                    ((Number) corps.get("livreur_id")).longValue(),
                    (String) corps.get("livreur_nom"),
                    ((Number) corps.get("eta_minutes")).intValue()));
        } catch (HttpClientErrorException.Conflict e) {
            return Optional.empty();    // 409 -> meme semantique qu'avant
        }
    }
    // liberer() : DELETE ; tous() : GET /livreurs -- a vous.
}
```

Point d'attention

L'hygiène du réseau, non négociable. Chaque appel sortant a un **timeout explicite** (ici 2 s / 3 s via la request factory) — sans lui, une panne du service Python *gèlerait* les threads du monolithe au lieu d'échouer proprement. Et le `RestClientException` restant (timeout, connexion refusée) doit être traduit par le `GestionnaireErreurs` en **503** : `{"erreur": "Service livraison indisponible, reessayez"}` — la troisième issue de l'appel réseau, nommée et gérée.

3.2 Les profils : l'interrupteur du Strangler

— Profil par défaut (`!distribue`) : l'implémentation locale — **les 10 juges restent verts sans rien démarrer**, la CI ne connaît pas Docker ;
— Profil `distribue` (+ propriété `livraison.url=http://localhost:8001`) : la façade HTTP. Démarrez le service Python, puis le backend avec `SPRING_PROFILES_ACTIVE=distribue` : passez une commande, confirmez — **le livreur vient du Python**, et `commandes` n'en sait rien.

Point cle a retenir

Relisez la figure du Strangler : vous êtes exactement en phase 2. Les deux chemins existent, l'interrupteur est un profil, le retour arrière est gratuit. C'est cela, migrer sans big bang — et c'est l'hexagone du Lab 2 qui, deux séances plus tard, paie encore : on a *swappé un adaptateur*, à l'échelle d'un processus entier.

Jalon Git

10 juges verts (profil défaut) ET parcours complet en profil `distribue` avec le service Python :

```
git tag lab4-etape2 && git push origin lab4-etape2
```

4 Etape finale — Compose, la carte... et la panne

Qui pilote cette section ?

L'Huissier pilote compose. **Le Rédacteur** pilote la carte. **Le Doyen** orchestre la panne finale au vidéoprojecteur — tout le groupe présent : c'est le moment fondateur du Lab 5.

4.1 docker compose : quatre conteneurs, un geste

`Dockerfile` du service Python (trois lignes de logique), puis le `docker-compose.yml` à la racine — le monolithe y reçoit son profil et l'adresse du service **par son nom** :

```
services:
  postgres:
    image: postgres:16
    environment: [POSTGRES_DB=senresto, POSTGRES_USER=senresto,
                  POSTGRES_PASSWORD=passer123]
    volumes: ["./backend/init-postgres:/docker-entrypoint-initdb.d"]
  livraison-service:
    build: ./livraison-service
  backend:
    build: ./backend
    environment:
      - SPRING_PROFILES_ACTIVE=distribue,postgres
      - LIVRAISON_URL=http://livraison-service:8001
    depends_on: [postgres, livraison-service]
    ports: ["8080:8080"]
  frontend:
    build: ./frontend
    ports: ["5173:5173"]
```

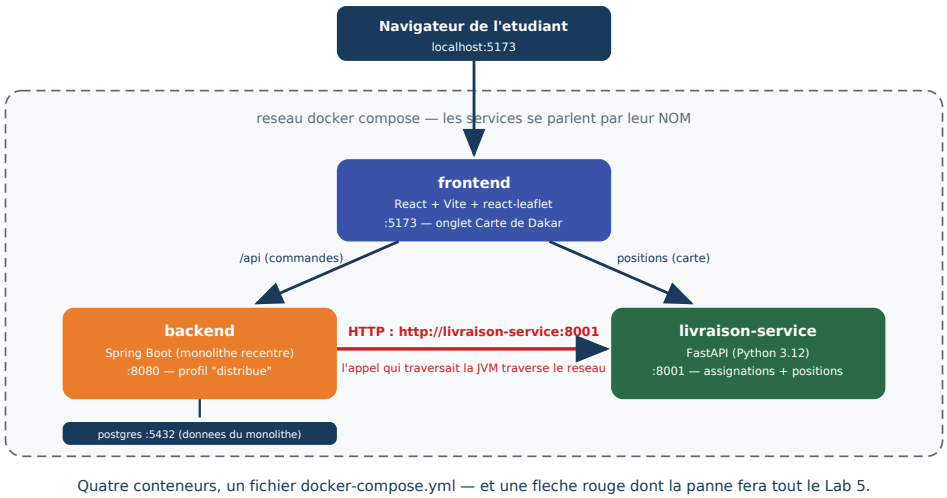


FIGURE 2 – L'orchestre : le nom du service est son adresse sur le réseau compose.

4.2 La carte de Dakar

```
cd frontend && npm install react-leaflet leaflet
```

Le Rédacteur ajoute l'onglet « Carte » : un composant `CarteDakar` centré sur Dakar, les restaurants en marqueurs fixes, et — pour la commande sélectionnée — le livreur en marqueur mobile, dont la position est interrogée **directement auprès du service Python** toutes les 3 secondes :

```
// Le coeur du composant (extraits -- le squelette complet est fourni
// dans le zip de la seance) :
<MapContainer center={[14.70, -17.45]} zoom={13} style={{height: "480px"}}>
  <TileLayer url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png" />
  {restaurants.map(r => <Marker key={r.id} position={positionDe(r)} />)}
  {positionLivreur && <Marker position={positionLivreur} icon={iconeScooter} />}
</MapContainer>
```

```
// Le polling : toutes les 3 s, si une commande CONFIRMEE est suivie
useEffect(() => {
  if (!livreurSuivi) return;
  const minuteur = setInterval(async () => {
    const r = await fetch(`${URL_LIVRAISON}/livreurs/${livreurSuivi}/position`);
    setPositionLivreur(await r.json());
  }, 3000);
  return () => clearInterval(minuteur);
}, [livreurSuivi]);
```

Lancez tout : `docker compose up -build`. Commandez un mafé chez Aminata, confirmez, ouvrez la carte : **le scooter d'Ibrahima avance vers Ouakam**. Cette fonctionnalité entière — positions, interpolation, demain le trafic réel — vit dans le service Python : le bénéfice de l'extraction, à l'écran.

Exemple

Deux questions légitimes, deux choix assumés. *« Et l'adresse exacte du client ? »* — SénResto travaille à la granularité du **quartier** : la distance est calculée (vraie formule de Haversine, vraie vitesse) sur le trajet **livreur** → **restaurant** → **client**, chaque étape au *centroïde* de son quartier. Une adresse postale exigerait un service de géocodage externe : hors périmètre, et l'approximation est honnête. *« Le trajet atteint 100 % mais la commande reste EN_LIVRAISON ? »* — Oui : la simulation est visuelle, le changement de statut reste manuel. Notez l'agacement que cela provoque : *le service sait que la course est finie...* c'est exactement un événement candidat pour le Lab 5.

Point d'attention

L'état qui s'évapore — votre premier vrai problème distribué. Les commandes vivent dans PostgreSQL (persistant) ; les courses et disponibilités vivent **en mémoire du service Python**. Redémarrez `livraison-service` : les courses sont oubliées, tous les livreurs redeviennent disponibles... alors que la base affirme le contraire. Deux services, **deux vérités, aucune synchronisation**. Constatez-le (redémarrez le service pendant une course), notez-le dans l'ADR — et gardez la blessure ouverte : la réconciliation d'état par événements est un des grands sujets du Lab 5.

4.3 L'experience finale : la panne

Au vidéoprojecteur, cérémonieusement :

```
docker compose stop livraison-service
```

Puis, depuis le frontend : créez une commande (**ça marche** — le monolithe vit), et tentez de la **confirmer**. Observez : le 503 après 2 secondes (votre timeout), le message propre (votre gestionnaire d'erreurs). Constatez et notez :

— la mort de `livraison-service` a **amputé** la confirmation — couplage *temporel* : les deux doivent être vivants *au même instant* ;

— et pourtant les SMS, le paiement, eux, n'ont besoin d'aucune réponse immédiate... pourquoi devraient-ils dépendre de la disponibilité de qui que ce soit ?

Redémarrez (`docker compose start livraison-service`), confirmez : tout revient. La question reste, ouverte et brûlante — c'est le Lab 5.

Jalon Git

Compose complet fonctionnel, carte vivante, panne provoquée et documentée :

```
git tag lab4-final && git push origin lab4-final
```

5 Recapitulatif : qui a fait quoi

Rôle	Étape 1	Étape 2	Étape finale
Le Doyen	relecture du contrat	invariant `commandes` intact	la panne au projecteur, ADR
L'Huissier	—	les deux profils Spring	`docker-compose.yml`
Le Rédacteur	—	—	l'onglet Carte (react-leaflet)
Le Notaire	contrat HTTP vs API module	`AdaptateurLivraisonHttp` + 503	vérif. messages d'erreur
L'Appariteur	`main.py` FastAPI + curl	—	`Dockerfile` du service

6 L'ADR du jour

L'**Architecte du jour** signe l'**ADR n°4 : « Pourquoi extraire Livraison — et pourquoi pas les autres ? »** (`docs/adr/adr-004-extraction.md`). Exigence nouvelle : une section *non-décisions* qui passe les six autres modules en revue et justifie leur maintien. Alternatives sérieuses : tout garder en modulith (statu quo défendable !) ; extraire en Java plutôt qu'en Python. Conséquences négatives à assumer : la latence, la troisième issue, deux stacks à maintenir, le déploiement qui exige compose.

7 Livrable et criteres de reussite

Critère	Vérification
Les 4 tags du lab présents	`git tag`
10 juges verts en profil défaut (sans Docker ni Python)	`./mvnw clean test`
Le service répond aux 4 endpoints, 409 compris	`curl`
`commandes` : zéro ligne modifiée	`git diff` ciblé
Timeouts explicites + 503 traduit	relecture + panne
`docker compose up` : parcours complet, carte vivante	démonstration
La panne provoquée, observée, réparée	historique + notes
ADR n°4 avec sa section non-décisions	`docs/adr/`

8 Vers le Lab 5 : ne plus mourir ensemble

Vous avez vu une commande refuser d'être confirmée parce qu'un *autre* processus était éteint. Pour l'assignation, ce couplage temporel est presque légitime — il faut une réponse. Mais les SMS ? Le paiement ? Informer n'exige pas que le destinataire soit réveillé : il suffit que le message **attende quelque part**. Ce quelque part s'appelle un *broker* — RabbitMQ — et au Lab 5, vos événements du Lab 3 traverseront le réseau par lui : notifications et paiement deviendront des consommateurs qu'on peut éteindre, rallumer, et qui rattrapent leur retard sans perdre un seul message. L'apothéose du semestre. Votre `lab4-final` en sera le point de départ.

Fin du Lab 4 — Dr. El Hadji Bassirou TOURE — DMI/FST/UCAD